

정답키 검증 프로젝트 — 회고

왜 오래 걸렸나 · 어떤 길로 실패했나 · 반복에 얼마가 들었나 · 어떻게 닫았나

전기기사 에이전트(ee-agent) · 5,331문항 · 작성 2026-06-16 · 모든 수치는 git·감사기록 (a) 출처

이 문서는 "무엇을 골랐나"만 적힌 결정 원장(decision records)에, 그 결정에 이르기까지의 **시행착오·실패 경로·반복 비용**을 더해 사람이 읽을 수 있는 서사로 복원한 회고다. 한 문장으로: **저장된 정답키의 4건 중 1건이 틀렸다는 측정에서 시작해, 약 두 달간의 반복 끝에 "전부 옳음을 증명한다"는 불가능한 목표를 "모든 문항에 상태를 배정한다"는 닫을 수 있는 목표로 재정의함**으로써 끝냈다.

~2개월

verify-agent 탄생(4-11)
→ 닫힘(6-14)

24.1%

저장 정답키 표본 오류율
(CI 16.4-34.1)

5,331

전건 종결 · limbo 0

543

5·6월 작업 commit
(250+256+a)

읽는 순서: ① 전체 타임라인으로 "오래"의 크기를 본다 → ② 시스템이 무엇을 골랐는지(6개 판단결정) → ③ 왜 오래 걸렸는지 다섯 가지 깊은 이유 → ④ 실제 실패 경로 카탈로그(되돌린 길들) → ⑤ 반복 비용의 정량 → ⑥ 무엇이 끝을 냈나 → ⑦ 남는 자산(방법론).

1. 타임라인 — "오래"는 얼마나 오래였나

검증은 단발 작업이 아니라 두 달에 걸친 **총총의 격투**였다. 작업 강도(월별 commit 수)만 봐도 4월 37 → 5월 250 → 6월 256으로 가속됐다. 그러나 진짜 시작점은 더 이르다 — 데이터 자체는 이 검증이 시작되기 **몇 달 전**에 OCR·자동 추출로 만들어졌고, 그때 검증 통로가 없었다는 사실이 이 모든 일의 뿌리다.

2026-04-11

verify-agent 탄생. "주장은 증거가 아니다"를 시스템과 Claude 자신에게 적용하기 시작. 동시에, 검증 대상이 된 문항 데이터는 이미 OCR/자동추출로 누적돼 있었다(검증 통로 없이).

2026-05

도구·앵커 구축기 (250 commit). lens-router 3띠 삼각측량, method-card, 무결성 게이트, 결정 원장(supervisor log) 등 "신뢰할 닷"을 짓는 데 한 달.

2026-06-08

전환점: 키 오류율 24.1% 측정. 표본 100건에서 저장된 human 답의 약 4건 중 1건이 오답으로 확인됨. 그동안 암묵적으로 깔려 있던 "키는 맞다"는 가정이 무너졌다. → 이후 모든 작업이 "옳음 증명"에서 "상태 배정"으로 강제 재편.

2026-06-08 ~ 13

대량 복원·재판독기 (256 commit의 대부분). 깨진 보기(choices) 복원 47 batch, 사각지대 전수검수 95 배치, 자율 재판독 최고 34차. 손으로 갈아낸 기간.

2026-06-13 ~ 14

통합·감사. 흩어진 검증 신호를 단일 사이드카(answer_trust.json)로 통합. 풀이↔답 대조 3,986건 무인 audit으로 고신뢰 불일치 100건 식별.

2026-06-14

단힘. 결정론 함수로 전 5,331건을 5상태 배지로 종결(limbo 0). "단힘 = 옳음 증명"이 아니라 "단힘 = 상태 배정"으로 재정의. 검증 트랙 종결.

2. 시스템이 무엇을 골랐나 — 여섯 개의 판단결정

아래는 결정 원장(decision records)에 기록된 6개의 핵심 판단을, 사람이 읽기 좋게 자연어로 푼 것이다. 각 결정에는 **버린 대안**이 있었다 — 이것이 기계 작업(단순 반복)과 판단을 가르는 기준이다.

결정 1 · 스코프의 출발점

2026-06-08 · 222b5c6

저장된 정답키를 "검증 대상"으로 격하한다 (맞이 아니다)

구작 human 답이 정답으로 저장돼 있었지만 그 정확도는 한 번도 측정된 적이 없었다. 무편향 표본 100건을 독립 검증한 결과 판정가능 87건 중 21건 불일치 = **오류율 24.1%**(Wilson 95% CI 16.4~34.1%). 직접 크롭 확인한 5건은 전부 실오답. 결론: 키를 진실의 답으로 쓸 수 없다.

왜 중요한가: 이 측정이 이후 모든 달기 판단의 전제다. "키가 맞다"는 가정을 폐기했기에, 달는 방식이 "전건 옳음 증명"이 아니라 "상태 배정"으로 갈 수밖에 없게 됐다.

버린 대안: 키를 그대로 신뢰(24.1%가 반증) · 전건 재검증(5,331건 멀티모달 재판독 = 무한 루프)

검증 신호를 단일 사이드카로 통합하되, 정답은 한 글자도 건드리지 않는다

검증 신호가 여러 사이드로(answer_source·비전 이중확인·사용자 검토)에 흩어져 같은 문항이 사이로마다 다른 상태였다. 이를 별도 파일 `answer_trust.json` 으로 통합하되 `questions.json` 의 답은 불변으로 뒀다.

왜 사이드카인가: 신뢰 메타를 정답 데이터와 분리하면, 정답을 수정하지 않고도 신뢰 상태만 앱에 표시할 수 있다. 검증 트랙이 정답 교정 트랙으로 번지는 것(히드라)을 막는 구조적 차단.

버린 대안: `questions.json`에 신뢰 필드 직접 추가(데이터·메타 혼합, diff 폭발 위험) · 사이로 유지(SoT 분기 미해소)

풀이↔답 대조는 구독 토큰으로, audit-only로 — LLM에게 교정 권한은 없다

미검증 4,688건의 95%가 미탐사였다. 풀이가 저장 답과 다른 결론을 내는 "고신뢰 불일치"를 찾으려면 대규모 LLM 대조가 필요했다. 이를 구독 토큰(`claude -p`, 유료 API 0)으로 3,986건 완주하되, 발견은 큐에 적재만 하고 자동 교정은 금지했다.

왜 audit-only인가: LLM에게 mutation 권한을 주면, 24.1% 오류율을 가진 또 다른 비검증 데이터가 누적된다(뿌리 함정의 재발). LLM은 "다른 답을 결론냈다"는 산출물만 내고, 교정은 사람 큐(needs_human 867)로 넘긴다.

버린 대안: 유료 API로 자동 교정(비검증 누적) · 표본만 audit(95% 빈곳이 남아 답힘이 거짓 마침표)

전 5,331건을 결정론 함수로 5상태 배지에 배정한다 (limbo 0)

audit 후에도 4,688건이 단일 '미검증'으로 뭉쳐 있었다. 모든 입력에 정확히 하나의 출력을 주는 **total function**으로 분해해, 구조적으로 미분류(limbo)가 0이 되게 했다 — 이것이 "담힘"의 코드 율타리다.

공식검증 535

교차일치 1,734

미검증 2,114

의심 50

손상 898

왜 5상태인가: '미검증' 한 덩어리를 의심(풀이가 다른 답)·손상(데이터 깨짐)·순수 미검증으로 나누면 학습자가 신뢰 수준을 한눈에 본다. 연속 점수(0~1)는 24.1% 오류율 데이터에 가짜 정밀도라 버렸다.

버린 대안: 단일 미검증 유지(limbo 잔존) · 연속 신뢰 점수(과적합)

"단힘"을 재정의한다 — 옳음 증명이 아니라 상태 배정

단힘을 "모든 문항이 종결 상태(**terminal disposition**)를 가짐"으로 정의했다. 미개봉 lane인 `source_mismatch`(stem↔crop 불일치 탐지)는 `not_computed` 로 명시 유예하고, 검증 트랙을 종결했다. HARD 규칙: 새 일괄 검증 = 히드라 재발.

왜 재정의가 끝을 냈나: 24.1% 오류율 데이터에서 "전건 옳음 증명"은 멀티모달 전수 재판독(무한 루프)을 요구하므로 거짓 마침표가 된다. `source_mismatch`를 열면 새 탐지 파이프라인 = 새 머리. 정직하게 비워두고, 공부 중 표면화되면 그것만 lazy 처리한다.

버린 대안: 전건 옳음 증명(무한 루프) · `source_mismatch` 개봉 후 닫기(닫기를 무한 연기)

손상 문제는 기본 제외하되 토글로 복원 가능 — 정보층과 선택층을 분리

손상 898건을 학습 회전에서 기본 제외(학습 대상 **4,433**)하되, 상단 토글로 복원 가능하게 했다 (`localStorage` 유지). 배지(신뢰 정보)와 덱 필터(노출 선택)를 두 층으로 분리했다.

왜 영구 숨김이 아닌가: 강제 숨김은 손상 898건을 영원한 사각지대로 고착시킨다. 토글은 사용자가 직접 확인·복구하는 lazy 교정의 진입점을 남긴다. 제외는 런타임 필터일 뿐 데이터 삭제가 아니다(app/data 무수정).

버린 대안: 손상 포함(깨진 데이터가 학습 흐름 희석) · 영구 숨김(교정 진입점 소멸)

3. 왜 오래 걸렸나 — 다섯 가지 깊은 이유

원장에는 결정만 있고 "왜 두 달이나"는 없다. 진짜 이유는 다섯 층으로 겹쳐 있었다. 위로 갈수록 표면, 아래로 갈수록 근본이다.

① 뿌리 — 데이터가 검증 통로 없이 태어났다 (가장 근본, 가장 늦게 보임)

이 모든 일의 -2층 원인. 문항 데이터는 OCR·자동 추출로 만들어졌는데, 그때 "생성과 동시에 검증"하는 통로가 없었다. 그래서 OCR 잡음·마커 오류·키 오류·보기 손상 같은 검증 안 된 가정이 몇 달간 조용히 누적됐다. 검증 프로젝트는 본질적으로 이 누적분을 사후에 닦아내는 작업이었고, 사후 정리 비용은 누적량에 비례한다. 한 건씩 보면 작은 문제("OCR이 글자 하나 틀림")지만, 5,331건 누적되면 시스템 신뢰도 자체의 문제가 된다. 가장 근본적인 함정이 가장 늦게 — 시스템이 두 달 돌아간 뒤에야 — 보였다.

② 히드라 — 머리 하나를 자르면 둘이 자란다

검증 lane을 하나 열 때마다 하위 문제가 새로 생겼다. **키 오류**를 보려 하니 → **보기(choices) 손상**이 드러나고 → 보기를 고치려니 **마커 번호 오류**가 나오고 → 그걸 보니 **규정의 버전 드리프트**(2021년 전후 KEC 개정)가 걸리고 → 그 너머에 **stem↔crop 불일치**(source_mismatch)가 있었다. 한 트랙을 "닫았다" 싶으면 다음 트랙이 열렸다. 닫기 재정의(결정 5)가 source_mismatch lane을 명시적으로 "열지 않는다"고 못 박은 이유가 바로 이것 — 더 자르지 않기로 결정해야 끝이 났다.

③ 인식론의 재발 — 같은 실수가 모든 층에서 반복됐다

같은 종류의 오류("주장 ≠ 증거", "요약 ≠ 본문", "측정 ≠ 결론")가 데이터·도구·추론의 모든 층에서 되풀이됐다. OCR 데이터가 그랬고, 크로스 에이전트(Codex)의 출력이 그랬고, Claude 자신의 추론도 그랬다. verify-agent의 사례집이 22건까지 쌓인 것 자체가 이 재발의 증거다. 글로 차단 조항을 적어도 자동 번역이 생기지 않아서, 매 재발마다 한 번의 교정 사이클이 들었다. 검증 프로젝트가 길어진 큰 이유는 "검증 시스템이 자기 자신을 검증하는 데" 계속 비용이 들었기 때문이다.

④ 수렴/발산 혼동 — 가짜 마침표가 무한 루프보다 위험했다

같은 프로젝트 안에서 작업의 성격이 계속 바뀌었다. 데이터 정제·복원은 **수렴**(끝을 숫자로 쓸 수 있음)이지만, 판단·프레이밍은 **발산**(취향·해석)이다. 발산 작업에 수렴 도구(끝-술어 done==true)를 들이대면 **가짜 마침표**가 생긴다. 실제로 규정 키 정제에서 "버전 드리프트를 잡았다"고 마침표를 찍었다가, 그 판정 자체가 틀렸음을 발견하고 통째로 **철회**한 일이 있었다(아래 4-A). 가짜 마침표는 되돌리는 데 무한 루프보다 더 비싸다 — 이미 그 위에 쌓은 것을 다 걷어내야 하기 때문.

⑤ 신뢰 제로의 비용 — 답을 직접 지어야 했다

키를 믿을 수 없으니, 키 없이 검증할 **독립 답**을 처음부터 지어야 했다. 사람 라벨 정답키, lens-router 3띠 삼각 측량, 풀이↔답 교차 대조 — 이 답들을 만드는 일 자체가 5월 한 달을 먹었다. "아무것도 믿지 않는다"는 원칙은 옳지만, 그 원칙을 지키려면 믿을 것을 손수 만들어야 하고, 그게 시간이다.

다섯 층을 한 줄로: 검증 안 된 채 태어난 데이터(①)를, 서로 얽힌 결함(②)과 모든 층에서 재발하는 인식론 오류(③) 속에서, 가짜 마침표의 함정(④)을 피하며, 믿을 답을 직접 지어가며(⑤) 닦아내야 했다.

4. 실패 경로 카탈로그 — 실제로 되돌린 길들

오래 걸린 시간의 상당 부분은 "틀린 길로 갔다가 되돌아온" 시간이다. 아래는 git에 흔적이 남은 실제 실패 경로들이다. 각각: 무엇을 했나 → 왜 틀렸나 → 비용 → 교훈.

실패 4-A · 철회

규정 키 "버전 드리프트 오판" — 판정했다가 통째로 철회

무엇을: 규정 과목 4개 분쟁을 "전부 키 오류"로 판정하고 마침표를 찍었다. 8fbd6f6 / 691fe23

왜 틀렸나: 현행 KEC 책으로 2021년 이전 기출의 답을 단정했다 — 시점 착오(version anachronism). 규정은 개정되는데, 개정 후 기준으로 개정 전 답을 틀렸다고 본 것. a1d0d4a [철회]

비용: 판정 1사이클 폐기 + 재검증 사이클 검색으로 3개 키오류가 버전 안정값임을 재확인하고 SUSPEND 복원. 1bbdac0

교훈: 표면 특성(현행 규정)을 본질(시점별 정답)으로 비약. 발산 작업에 성급한 끝-술어를 쓴 전형.

실패 4-B · 메트릭 오설정 자각

2-레이어 예측 모델 — 만들고 보니 메트릭이 틀려 있었다

무엇을: 데이터 품질을 예측하는 2-레이어 모델 + 30문항 예측 테스트를 설계했다.

왜 틀렸나: 예측이 확인되지 않았고, 측정 지표 자체가 잘못 설정돼 있었음을 스스로 자각. c75c508

비용: 실험 1세트 폐기 "측정한 것이 묻는 질문에 답하는가"를 사전에 못 묻은 비용.

교훈: 측정의 방향이 질문의 방향과 어긋나면, 아무리 정교한 측정도 버려진다.

실패 4-C · 가짜 경보

"충돌 20건" 경보 — 알고 보니 stale 스냅샷과 비교했을 뿐

무엇을: 파일럿 30건 비전 전사에서 "공식정답과 20건 충돌"이라는 경보를 냈다. a7dcb6c

왜 틀렸나: 현재 데이터가 아니라 오래된 스냅샷과 비교한 결과였다. 현재 데이터로 다시 보니 28/28 전부 일치. 0a4a0d2 [정정]

비용: 가짜 위기 대응 + 정정 사이클 없는 위기를 진단하느라 든 시간.

교훈: 비교의 기준(무엇과 비교했나)을 (a) 출처로 확인하지 않으면, 측정이 정직해도 결론이 거짓이 된다.

실패 4-D · 반복되는 도구 오류

크로스 에이전트(Codex)의 마커 번호 오류 — 한 번이 아니라 계속

무엇을: Codex에게 보기/마커 추출을 위임하고 그 산출을 받았다.

왜 틀렸나: Codex가 마커 번호를 반복해서 틀렸다 — 309/311/293, 그 다음 309 또(2번째), 563(3번째)··· 매 batch마다 재발. 9373cdd · f4001e8 · 12444fa

비용: 47 batch 전부 lead 재판독 Codex 산출을 그대로 믿을 수 없어, 매 배치를 사람(lead)이 크롭 직접 확인으로 재검증해야 했다.

교훈: 위임의 출력은 "결론"이 아니라 "검증할 산출물"이다. 독립 재판독이 매번 측정을 구했다(\$1.8의 실증).

실패 4-E · 직렬화 결함

한 글자 고치려다 diff가 293,834줄로 폭발

무엇을: 검증된 답 1건(2018_1회_14, 2→1)을 적용하며 JSON을 저장했다.

왜 틀렸나: 의미 변경은 1건인데, 저장 시 직렬화 포맷(compact→pretty)이 바뀌어 파일 전체가 재포맷됐다. 0f4a600 · 5024b21 (CONSTITUTION 사례 22)

비용: 나쁜 commit reset + 포맷 보존 추가 + 재적용 diff 입자가 의미 입자와 어긋나 git blame·리뷰·부분 롤백 가치를 잃을 뻔.

교훈: 의미 scope(1건)와 diff scope(파일 전체)는 다른 축이다. 적용 전 직렬화 포맷을 감지·보존해야 한다.

이 실패들의 공통 구조

4-A부터 4-E까지 전부 "검증 안 된 다리" 위에 결론을 세웠다 — 현행 규정으로 과거를 판정(A), 방향 틀린 측정(B), 기준 모를 비교(C), 믿을 수 없는 위임 출력(D), 무시한 직렬화 포맷(E). verify-agent의 사례 집(0~22)은 바로 이 다리들의 카탈로그이고, 그것이 두꺼워진 만큼이 "오래"의 정체다. 실패가 비용이 된 게 아니라, 실패를 기록으로 바꾼 것이 자산이 됐다.

5. 반복 비용의 정량 — 손으로 갈아낸 자국

아래는 git에 남은 반복의 실측이다. "추정"이 아니라 commit 수 직접 집계.

반복 트랙	규모	무엇이었나
자율 재판독 (null LOCAL)	최고 34차	같은 미검증 문항 풀을 매 차수 다시 읽으며 lead 검증으로 교정. 27개 차수 라벨이 commit에 남음.
보기(choices) 복원 batch	47 commit	footer 손상·이중확인·crop 게이트·codex_fail 등 여러 트랙으로 깨진 보기를 batch 단위로 복원/보류.
사각지대 전수검수	95 배치 / 10 라운드	아무도 안 본 빈곳을 95개 배치로 쪼개 이중확인. 라운드마다 1~9 건씩 교정.
세션 핸드오프·재개 문서	11 문서	세션 경계마다 컨텍스트를 다시 읽는 비용. 한 번의 인계 = 한 번의 재컨텍스트.
전체 작업 강도 (5~6월)	~506 commit	5월 250 + 6월 256. 이 중 상당수가 위 반복 트랙.

이 표가 말하는 것: 검증은 한 번의 똑똑한 판단으로 끝나지 않았다. **똑똑한 판단(6개) + 무수한 손작업(수백 commit)**의 합이었고, 손작업이 길었던 이유는 §3에서 본 다섯 총 때문이다. 그리고 이 손작업의 대부분은 **한 번 더 일괄 검증을 돌리면 그대로 재발한다** — 그래서 닫힘 규칙이 "새 일괄 검증 금지(히드라)"를 HARD로 못 박았다.

6. 무엇이 끝을 냈나 — 발산을 수렴으로 바꾼 한 번의 재정의

두 달을 끈 일이 며칠 만에 닫힌 이유는 새 노동이 아니라 **목표의 재정의**였다.

전환

"전부 옳음을 증명한다" → "모든 문항에 상태를 배정한다"

"옳음 증명"은 끝을 가짜 없이 쓸 수 없는 **발산 목표**다(24.1% 오류 데이터에선 전수 멀티모달 재판독 = 무한 루프). 이를 "상태 배정"이라는 **수렴 목표**로 바꾸자, 모든 입력에 하나의 출력을 주는 결정론 함수 (total function)로 **limbo가 구조적으로 0**이 됐다. 발산을 수렴으로 변환하는 순간, 닫을 수 있는 끝-술어가 생겼다.

닫힘의 정의를 바꾼 것이 핵심이다. **닫힘은 옳음의 증명이 아니라 상태의 배정**이다. 모든 문항이 종결 상태를 갖는 순간 프로젝트는 닫힌다 — 그 답이 전부 맞다는 뜻이 아니라, 더 이상 미분류가 없다는 뜻이다. 나머지(의심·미검증)는 공부하다 만나면 그것만 lazy 교정한다.

7. 남은 자산 — 방법론

이번 프로젝트의 가장 값진 산출물은 정제된 데이터가 아니라, 비개발자가 LLM으로 개발할 때 무한 루프·블랙박스를 끊는 원리다.

자산 1

세 개의 코드 울타리 — LLM은 안에서, 코드는 경계에서

① **끝-술어**(`done==true` , 멈춤 기준): LLM에 맡기면 "완료"를 거짓 선언한다. ② **불변식 가드**("안 건드릴 X 증명", md5/diff): 가장 보편적, 거의 어디나 유효. ③ **상태 파일**(단일 JSON, 산문 아님): 255개 문서 사일로의 해독약. → LLM은 **추출·생성·판단**(울타리 안), 코드는 **끝·불변식·상태**(울타리). 둘 사이의 막(膜)은 **스키마** — LLM은 산문이 아니라 JSON으로 내고, 코드가 검증한다.

자산 2

수렴 vs 발산 — 울타리의 적용 경계

수렴(끝을 가짜 없이 숫자로 쓸 수 있음 — 정제·마이그레이션·검증): 세 울타리 전부 적용. **발산**(창작·연구·판단·취향): 끝-술어 금지(가짜 마침표가 무한 루프보다 위험), 대신 "반복 박스 + 사람 멈춤". 단 불변식은 유지. **한 프로젝트 안에서 종류가 바뀐다** → "지금 이 단계는 수렴인가 발산인가"를 자주 다시 물어라. (이번 프로젝트의 4-A 철회가 이 질문을 안 물어서 난 사고다.)

자산 3

두 앵커 레이어 — 사람의 것을 닮으로

Q1 판단 원장(supervisor log + decision record): "왜 골랐나"의 영속 기록. 판단 작업용(기계 작업엔 무의미). **Q2 독립 정답키 삼각측량**(lens-router 3띠): forward 물리량 도출이 깨지면(비가역), 사람 라벨 정답키를 닮으로 삼아 거꾸로 검증. 이번 단기의 **척추**. 두 레이어의 공통 원리: **사람의 것(판단·진실)**을 독립·영속 산출물로 박아 기계가 닮으로 삼는다.

이 회고 문서 자체가 Q1 원장의 빈칸을 메우는 행위다. 결정(무엇을)은 6월에 기록됐지만, 그 결정에 이르는 시행착오(왜·어떻게·얼마나)는 비어 있었다. 그 빈칸이 곧 "오래 걸린 시간"의 정체이고, 이제 기록으로 남았다.

출처 무결성: 본 문서의 모든 수치·날짜·실패 사례는 git commit 본문, `docs/audit/trust_audit/closure_report.json`, `docs/audit/decision_records/*.jsonl`, `verify-agent CONSTITUTION.md` 사례집에서 (a) 출처로 복원했다. 추정·외삽은 사용하지 않았다(사례 2·8 차단). 반복 비용은 표본 비율이 아니라 commit 직접 집계.

전기기사 에이전트(ee-agent) · 정답키 검증 프로젝트 회고 · 2026-06-16 · 닫힘=상태배정 / 코드는 울타리·LLM은 안 / 수렴엔 끝·술어·발산엔 반복박스 / 사람의 진실을 닮으로.